SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration :60 Mins
On Class

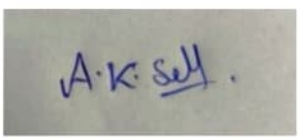
Total Marks : 50

Annexure A
Class Test

Code of Conduct

I A K Selva Pranhu(192421224)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



A.K. Selvapraleha
1924212 24

CLASS TEST

Q1. Write down the translation scheme to generate code for assignment statement. List the scheme for generating three address code for the assignment statement x = (a + b) * (c + d)

Q2.Consider the following Boolean expression: (a > b) && (c < d) || (e < f)

The grammar for Boolean expressions is given as:
B → B1 || B2 , B → B1 && B2 , B → ! B1 , B → E1 relop E2 , B → true , B → false, E → id

(i). Explain how operator precedence and associativity affect the evaluation of the given Boolean expression.

(ii). Construct the Three Address Code (TAC) for the given Boolean expression using short-circuit evaluation.

(iii). Represent the control flow of the Boolean expression using appropriate labels and conditional jump statements.

Q3.Given the declaration sequence

int a, b[10];

float c, d; derive the intermediate representation including symbol table entries, type sizes, and offset calculations.

Q4.Consider the following program fragments:

(i) . while (A < C && B > D) do
if (A == 1) then
C = C + 1
else
while (A <= D) do
A = A + B Generate the Three Address Code (TAC) for the above program fragment. Clearly show: (a) Conditional and unconditional jump statements, (b) Use of labels for control flow, (c) Evaluation of Boolean expressions

(ii). main()
{
int i;
int a[10];
i = 1;
while (i <= 10)
{
a[i] = 0;
i = i + 1;
} }

Translate the executable statements of the above C program into Three Address Code (TAC). Clearly indicate: (a) Representation of array elements, (b) Loop control using labels, (c) Increment and assignment operations

Q5. Use backpatching to generate intermediate code for:

```
if (a < b && c < d)
 x = a + b;
else
x = c + d;
```

Show the intermediate instructions before and after backpatching. (10 Marks)

Class Test

A.K. Selvaprabhu
19247212224

4:

(i) while (A<C && B>D) do
if (A==1) then
C=C+1
else
while (A<=D) do
A=A+B

100: if A<C then goto 102
101: goto 115
102: if B>D then goto 104
103: goto 115
104: if A=1 then goto 106
105: goto 109
106: t_1 := C+1
107: C := t_1
108: goto 100
109: if A<=D then goto 111
110: goto 100
111: t_2 := A+B
112: A := t_2
113: goto 109
114: goto 100
115: *

Boolean Evaluation:

A<C && B>D

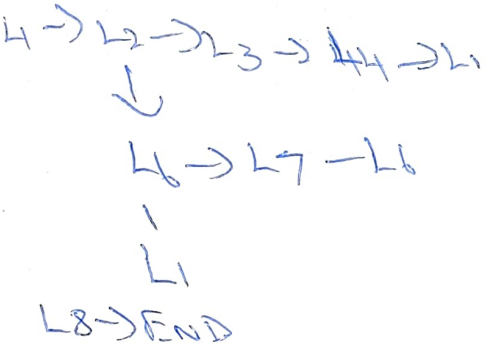
if A<C goto L2

goto L8

L2:
if B>D goto L3

goto L8

Control Flow:



(ii) Given: i = 1;
while (i<=10)
{ a[i] = 0;
i = i+1;
}

100: i := 1
101: if i<10 then goto 103
102: goto 108
103: t_1 := i

Array Representation:

Address(a[i]) = Base(a) + i x width(int)
width(int) = 4
t_1 = i * 4
a[t_1] = 0.

104: $t_2 := 4 * t_1$

105: $a[t_2] := 0$

106: $i := i + 1$

107: goto 109

108: stop

Loop Control

L1: if $i <= 10$ goto L2

goto L5

L2: $t_1 = i * 4$

$a[t_1] = 0$

$t_2 := i + 1$

$i = t_2$

goto L1

L5:

1). $x = (a+b) * (c+d)$

Translation Schme:

$S \rightarrow id = E$

$E \rightarrow E_1 + E_2$

{E.place :newtemp(); emit (E.place = I)

$E \rightarrow E_1 * E_2$

{E.place = newtemp(); emit (Eplace : I]

$E \rightarrow (E_1)$

{E. place = E_1. place }

$E \rightarrow id$

{E. place = id. lexeme }

Three address code:

$t_1 = a + b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$x = t_3$

2). Given: $(a > b)$

$(a > b) \&\& (c < d) || (e < f)$

operator Precedence

>, < > && > ||

$((a > b) \&\& (c < d)) || (e < f)$

Short circuit Tac

L1:

if $a > b$ goto L2

goto L4

L2:

if $c < d$ goto. L true

goto L false.

Ltrue:

result = true

goto Lend

Lfalse:

result = ~~bool~~ false

Lend:

Backpatch lists

(a > b):

truelist = {1}, false list = {2}

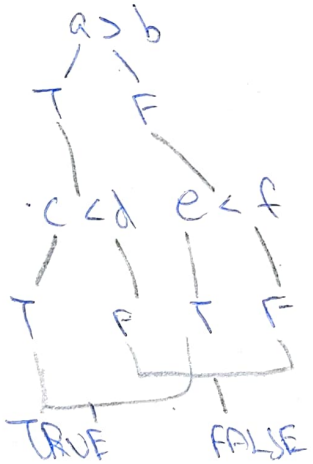
(c < d):

truelist = {3}, falselist = {4}

(e < f):

truelist = {5}, falselist = {6}

CFG



3) Given:

```
int a, b[10];
float c, d;
```

Assume:

sizeof(int) = 4 bytes

sizeof(float) = 4 bytes

Symbol table.

Name	Type	Width	Offset
a	int	4	0
b	array(10,int	40	4
c	float	4	44
d	float	4	48

Calculations.

width (a) = 4 — offset (a) : 0

width (b) = 10 x 4 = 40 — offset (b) : 0 + 4 = 4

width (c) = 4 — offset (c) : 4 + 40 = 44

width (d) = 4 — offset (d) : 44 + 4 = 48

Total width = 4 + 40 + 4 + 4 = 52 bytes.

Intermediate Representation

a : int

b : array [10] of int

c : float

d : float

5). Given:

```
if (a < b && c < d)
    x = a + b;
else
    x = c + d
```

Before Backpatching.

1. if a < b goto
2. goto
3. if c < d goto _
4. goto

5. $t_1 = a + b$
6. $x = t_1$
7. goto
8. $t_2 = c + d$
9. $x = t_2$

Lists.

$B_1 = (a < b)$

$B_1.truelist = \{1\}$

$B_1.falselist = \{2\}$

$B_2 = (c < d)$

$B_2.truelist = \{3\}$

$B_2.falselist = \{4\}$

For AND

backpatch($\{1\}, 2, 3$)

B.truelist $= \{3\}$

B.falselist $= \{2, 4\}$

After Backpatching

1. if $a < b$ goto L_1
2. goto L_3
4.
3. if $c < d$ goto L_2
4. goto L_3

L_2 :

5. $t_1 = a + b$
6. $x = t_1$
7. goto L_4

L_3 :

8. $t_2 = c + d$
9. $x = t_2$

L_4 :

Backpatching

backpatch ({1}, 3)

backpatch ({3}, 5)

backpatch ({2, 4}, 8)

Final TAC

L1: if a < b goto L2

goto L3

L2: if c < d goto L4

goto L3

L4: $t_1 = a + b$

$x = t_1$

goto L5

L3: $t_2 = c + d$

$x = t_2$

L5: